PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL
FORMATO GUÍA DE APRENDIZAJE – ACTIVIDADES
ESTRUCTURAR LA PROPUESTA TÉCNICA

Actividad

CONSUMO Y DOCUMENTACIÓN DE UNA API CON REACT

Presentado a: Instructora Alexandra Muñoz
Por Aprendiz: **Jenny Carolina Marrugo Ussa**
Ficha: 3312932

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día **24** de **octubre** del año **2026**

Introducción

Desarrollé una aplicación web en el lado del cliente utilizando React (con Vite) que se conecta y consume los servicios de una API REST previamente construida con Node.js y Express en el lado del servidor.Resuelve la necesidad de separar la interfaz gráfica (lo que ve el usuario) de la lógica de procesamiento y almacenamiento de datos (el backend), permitiendo consultar mensajes informativos en tiempo real y enviar datos de usuarios mediante formularios interactivos.

La comunicación se realiza mediante el protocolo HTTP a través de internet o entorno local (localhost). El cliente (React) hace peticiones de tipo GET o POST utilizando la función nativa fetch() apuntando a la URL del servidor (http://localhost:5000), mientras que el servidor (Express) procesa la solicitud, aplica validaciones y devuelve una respuesta estructurada en formato JSON, la cual es interpretada y mostrada en la pantalla por React.

Instalación y Configuración

- **Node.js y npm:** Node.js es el entorno de ejecución que nos permite correr JavaScript en el servidor (fuera del navegador). npm (Node Package Manager) es el gestor de paquetes utilizado para instalar Express, CORS y administrar las dependencias del proyecto.

- **Creación del proyecto con Vite**: Creé la aplicación Frontend ejecutando en la terminal npm create vite@latest, seleccionando la plantilla de React con JavaScript, lo que genera una estructura de carpetas ligera y muy rápida para el desarrollo.

- **Archivos principales:**

  main.jsx: Es el punto de entrada de React. Se encarga de enlazar la aplicación con el archivo index.html e inyectar el componente principal en el DOM.

  App.jsx: Es el componente raíz de la aplicación en el cual importé y organicé los tres componentes principales (Inicio, Informacion y Solicitudes).

- **Ejecución del Frontend:** Se inicia con el comando npm run dev, el cual levanta el servidor de desarrollo local (normalmente en http://localhost:5173)

- **Verificación del Backend**: Comprobé que Express estuviera activo ejecutando node app.js en la consola (observando la confirmación *"Servidor ejecutándose correctamente en el puerto 5000"*) y haciendo pruebas de acceso a las URLs en el navegador.

Desarrollo de los Componentes

- **Inicio.jsx:**

  - **Responsabilidad:** Consultar el mensaje de bienvenida inicial del sistema al presionar un botón.

  - **Endpoint & Método:** GET /inicio.

  - **Ejecución:** Se dispara mediante un evento onClick en un botón que ejecuta una función asíncrona con fetch().

  - **Manejo de errores:** Captura fallos con try/catch para informar al usuario si no fue posible entablar conexión con Express.

- **Informacion.jsx:**

  - **Responsabilidad:** Obtener y desplegar los datos generales de la asignatura y el módulo.

  - **Endpoint & Método**: GET /informacion.

  - **Ejecución:** Se ejecuta a solicitud del usuario mediante interacción con



botón.

- **Manejo de errores**: Almacena la respuesta en un estado y maneja excepciones en caso de respuestas no exitosas.

- **Solicitudes.jsx:**

  - **Responsabilidad**: Capturar mediante un formulario los campos de Nombre, Apellido y Correo, para enviarlos al backend y mostrar la confirmación.

  - **Endpoint & Método:** POST /solicitudes.

  - **Ejecución:** Captura los valores de los inputs mediante eventos de cambio y los envía empaquetados en un objeto JSON con Content-Type: application/json.

  - **Manejo de errores:** Valida si la respuesta del servidor no es ok (por ejemplo, por campos vacíos con código HTTP 400) o si hay un corte de conexión con el puerto 5000.

Explicación del Código y Conceptos Clave

- **fetch():** Función nativa de JavaScript utilizada para realizar peticiones de red asíncronas a una URL específica.

- **async / await:** Mecanismo que nos permite redactar código asíncrono de forma más clara y secuencial. async declara la función asíncrona y await detiene la ejecución en esa línea hasta que el servidor devuelva la promesa.

- **response.json():** Método que procesa la respuesta HTTP en texto plano y la convierte en un objeto manipulable de JavaScript.

- **JSON.stringify():** Convierte un objeto de JavaScript a formato de texto JSON, necesario para enviar datos en el cuerpo (body) de una petición POST.

- **useState:** Hook de React utilizado para crear y actualizar variables de estado dentro de los componentes. Al modificar un estado, React actualiza la vista automáticamente.

- **Manejo de eventos:** Funciones como onClick que reaccionan a las interacciones del usuario (clics en botones) para activar el consumo de la API.

Investigación Teórica Obligatoria

1. **¿Qué es una API y para qué sirve en una aplicación web?**

Es una interfaz o puente de comunicación que permite a dos sistemas (como React en el navegador y Express en el servidor) intercambiar datos de forma organizada.

2. **¿Qué diferencia existe entre frontend y backend?**

El Frontend es la interfaz gráfica que ve e interactúa el usuario. El Backend es la lógica que corre en el servidor, procesa peticiones y maneja las bases de datos.

3. **¿Qué significa consumir una API?**

Es hacer solicitudes de datos desde una aplicación cliente hacia una URL del servidor para leer, guardar o modificar información.

4. **¿Qué función cumple fetch()?**

Es la herramienta propia de JavaScript para enviar o traer datos a través del protocolo HTTP.

5. **¿Por qué se utilizan async y await en una petición HTTP?**

Se usan porque las respuestas de servidor toman un tiempo en llegar. Permiten esperar la respuesta sin bloquear el resto del programa.

6. **¿Qué es una promesa en JavaScript?**

Es un objeto que representa una operación que finalizará en el futuro, ya sea con éxito (resuelta) o con error (rechazada).

7. **¿Para qué sirve response.json()?**

Para traducir la respuesta codificada que nos manda el servidor a un objeto que React pueda leer e imprimir en pantalla.

8. **¿Qué diferencia existe entre una petición GET y una petición POST?**

GET se usa únicamente para consultar o pedir información. POST se utiliza para enviar datos nuevos al servidor (como los de un formulario).

9. **¿Qué función cumplen las cabeceras HTTP y por qué se utiliza Content-Type: application/json?**

Las cabeceras le dicen al servidor qué tipo de información le estamos enviando. Content-Type: application/json le aclara que los datos vienen en formato JSON.

10. **¿Por qué se utiliza JSON.stringify() al enviar datos en una petición POST?**

Porque el canal HTTP transmite texto. Con esta función convertimos el objeto de JavaScript en una cadena de texto JSON que el servidor sí entiende.

11. **¿Qué diferencia existe entre un error de conexión y una respuesta HTTP con código 400?**

El error de conexión ocurre si el servidor está apagado y la petición no llega. El código 400 ocurre cuando la petición sí llega, pero el servidor la rechaza por datos incompletos o mal formados.

12. **¿Qué función cumplen useState y los eventos onClick y onSubmit?**

useState guarda la información en variables reactivas, mientras que onClick y onSubmit disparan las funciones cuando el usuario interactúa con los botones o formularios.

13. **¿Qué es CORS y por qué puede afectar la comunicación entre React y Express?**

Es una regla de seguridad de los navegadores que prohíbe llamadas entre distintos puertos (como el 5173 de React y el 5000 de Express) si el backend no lo autoriza expresamente.

14. **¿Qué ocurre si el frontend envía propiedades diferentes a las que espera el backend?**

El backend no logrará leer las variables (llegarán como undefined) y las validaciones o registros de datos fallarán.

15. **¿Por qué no se debe considerar exitosa toda petición solamente porque fetch() devolvió una respuesta?**

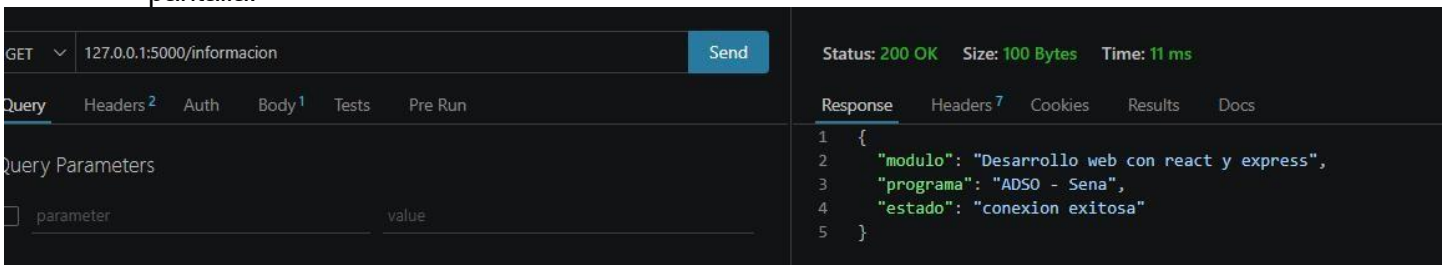
Porque fetch() solo falla si no hay red. Si el servidor responde con un error 400 o 500, para fetch() la llamada existió, por lo que debemos comprobar la propiedad response.ok

6. Pruebas y Resultados

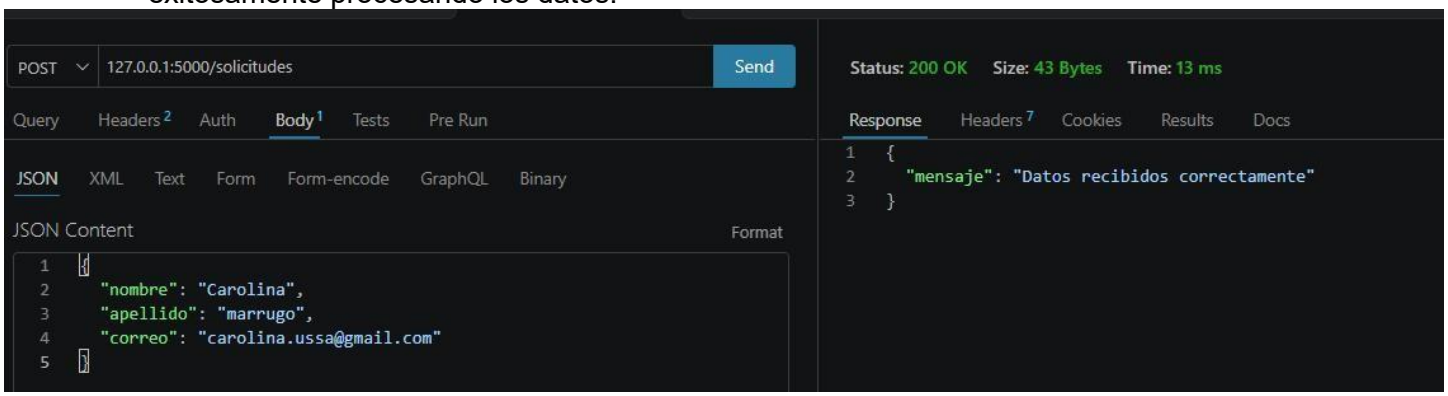
1. **Consulta /inicio:** Se verifica que al hacer clic en el botón, aparece el mensaje recibido desde Express y en la consola *Network* el código de estado es 200 OK.



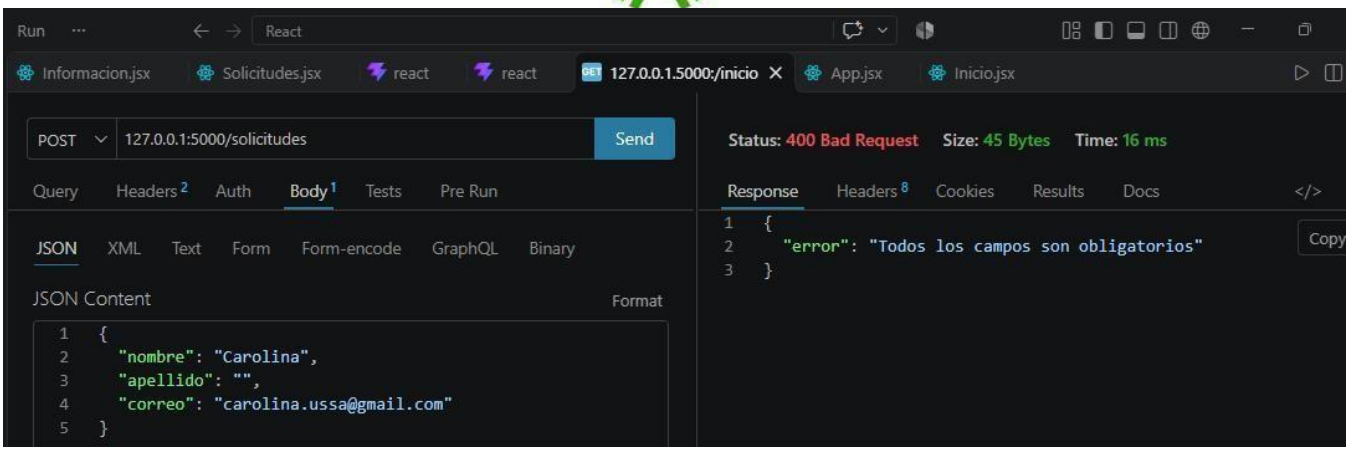
2. **Consulta /informacion:** Se confirma la recepción del objeto JSON con los datos del módulo en pantalla.

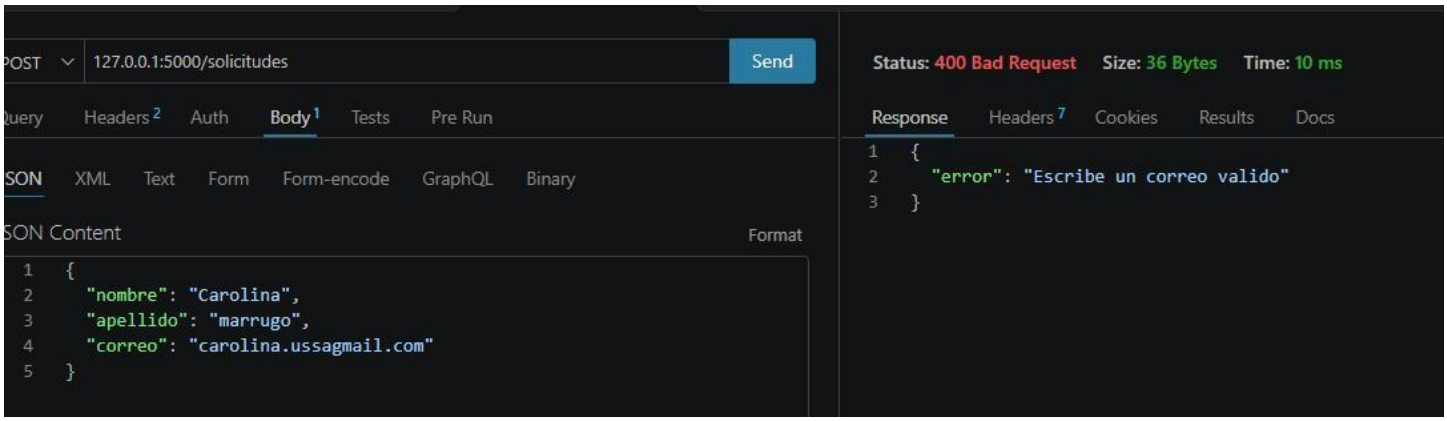


3. **Envío de Formulario Completo:** Se ingresan Nombre, Apellido y Correo; el servidor responde exitosamente procesando los datos.

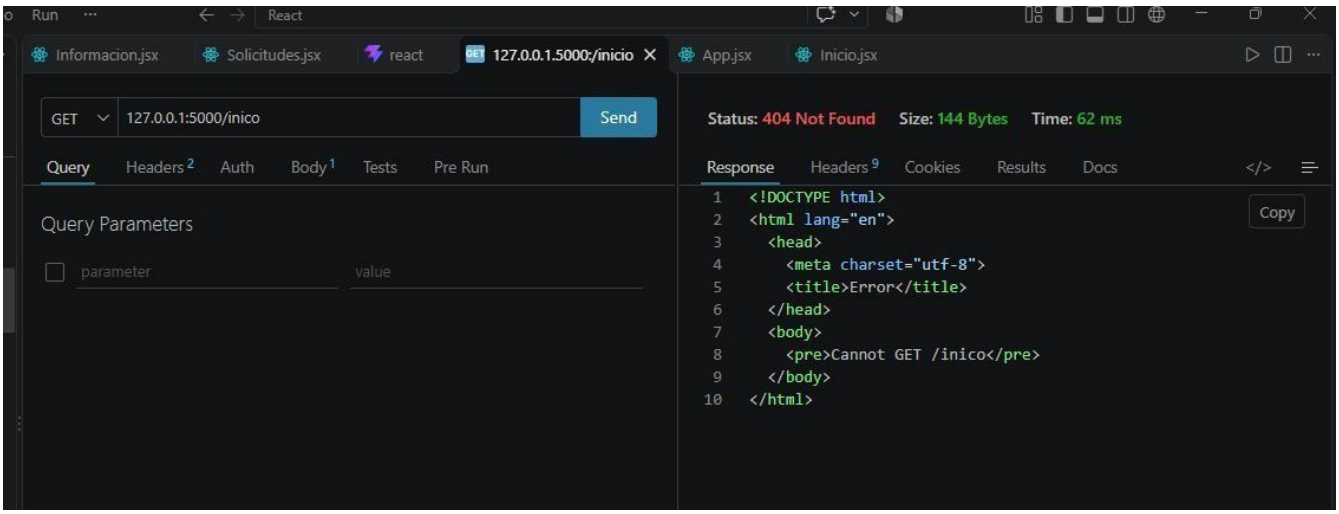


4. **Envío de Campos Vacíos:** Al enviar el formulario sin datos, el backend responde con código 400 Bad Request y la interfaz alerta que todos los campos son obligatorios.



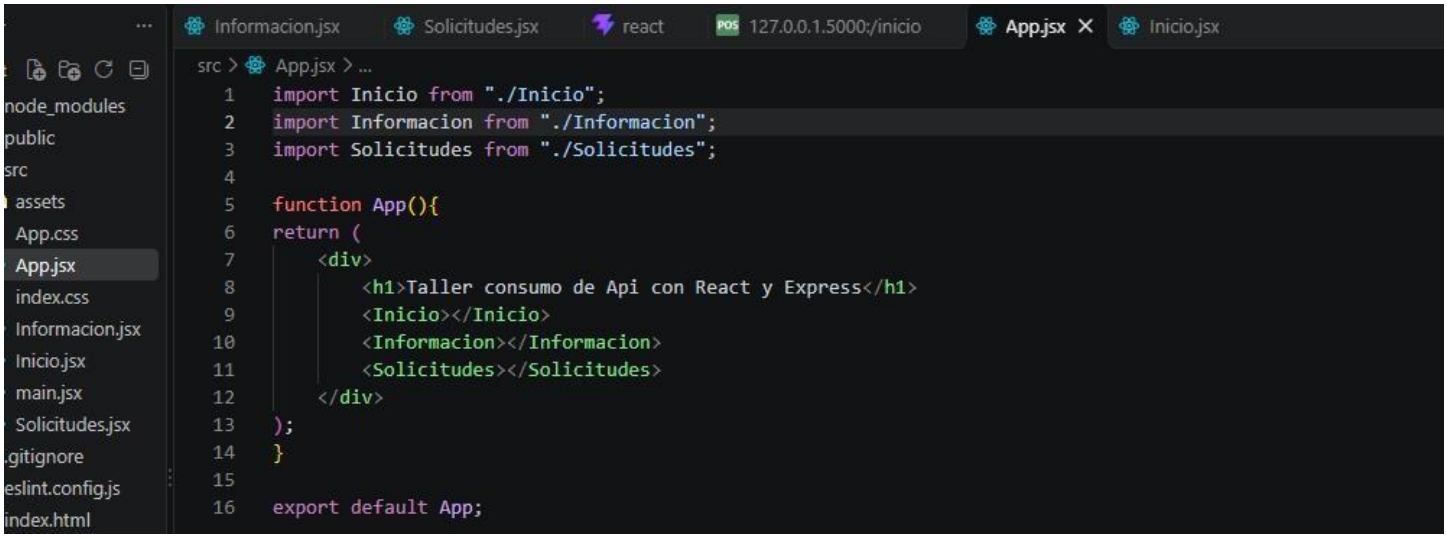


5. **Prueba con Servidor Apagado:** Se detiene Node.js en la consola y se presiona un botón; el bloque catch atrapa el error y despliega el mensaje de fallo de conexión.

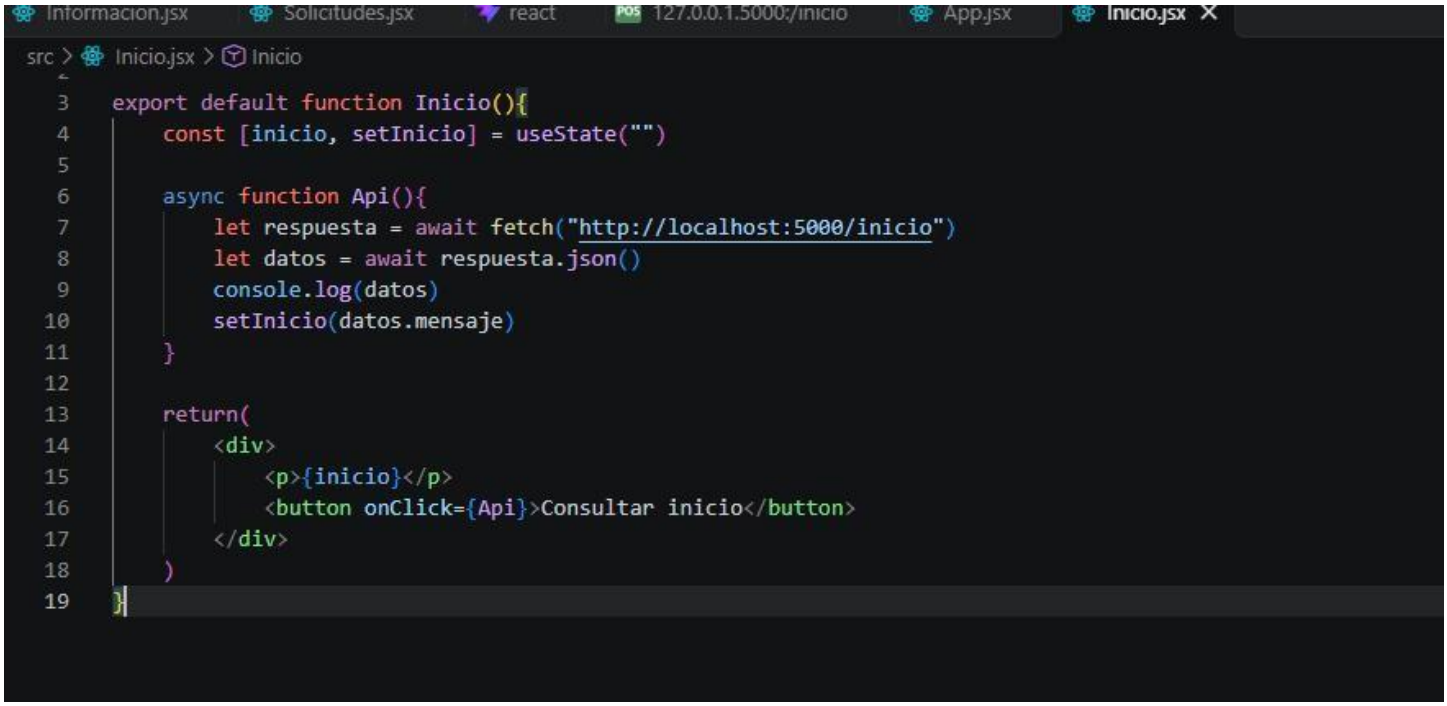


REACT

App.jsx



Inicio.jsx



Informacio.jsx

src > Informacion.jsx > Informacion

```jsx
import { useState } from "react"

export default function Informacion(){
    const [modulo, setModulo] = useState("")
    const [programa, setPrograma] = useState("")
    const [estado, setEstado] = useState("")

    async function Api(){
        let respuesta = await fetch("http://localhost:5000/informacion")
        let datos = await respuesta.json()
        console.log(datos)
        setModulo(datos.modulo)
        setPrograma(datos.programa)
        setEstado(datos.estado)
    }

    return(
        <div>
            <button onClick={Api}>consultar informacion</button>
            <p>{modulo}</p>
            <p>{programa}</p>
            <p>{estado}</p>
        </div>
    )
}
```

Solicitudes.jsx

src > Solicitudes.jsx > Solicitudes

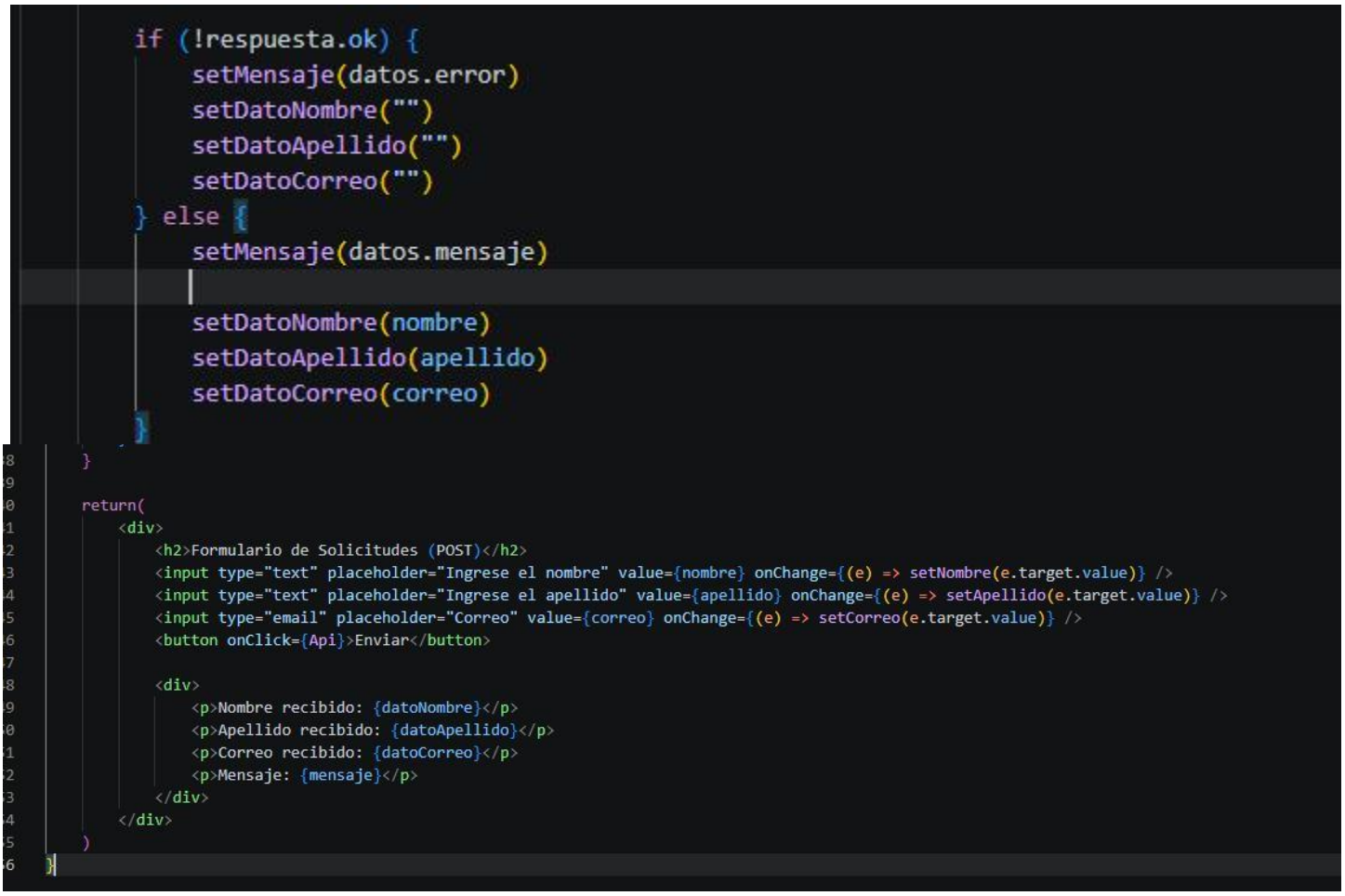
```jsx
import { useState } from "react"

export default function Solicitudes(){
    const [nombre, setNombre] = useState("")
    const [apellido, setApellido] = useState("")
    const [correo, setCorreo] = useState("")

    const [datoNombre, setDatoNombre] = useState("")
    const [datoApellido, setDatoApellido] = useState("")
    const [datoCorreo, setDatoCorreo] = useState("")
    const [mensaje, setMensaje] = useState("")

    async function Api(e){
        if (e) e.preventDefault()

        let respuesta = await fetch("http://localhost:5000/solicitudes", {
            method: "POST",
            headers: {
                "Content-Type": "application/json"
            },
            body: JSON.stringify({ nombre, apellido, correo })
        })
        let datos = await respuesta.json()
        console.log(datos)
```



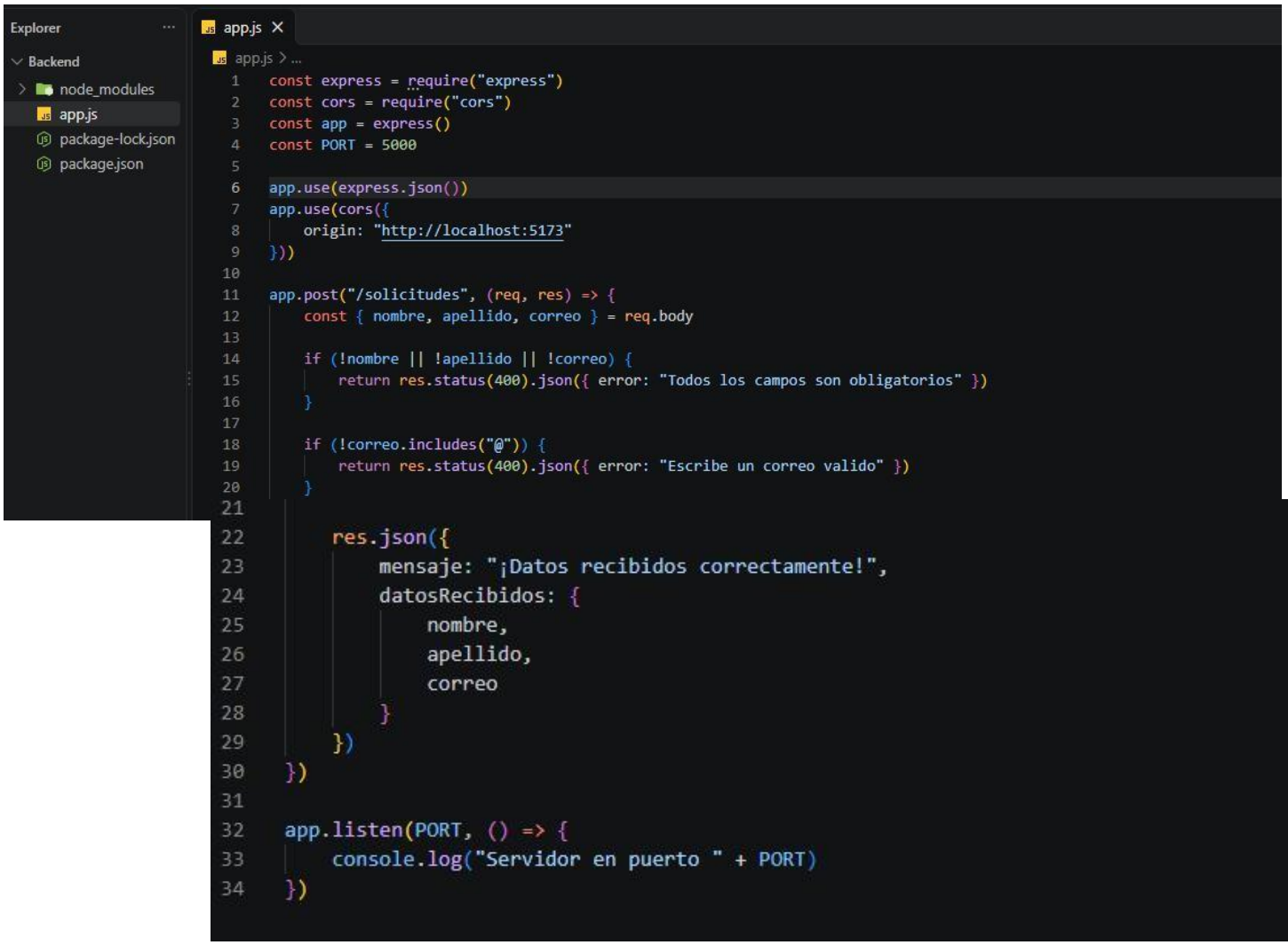
```
        if (!respuesta.ok) {
            setMensaje(datos.error)
            setDatoNombre("")
            setDatoApellido("")
            setDatoCorreo("")
        } else {
            setMensaje(datos.mensaje)

            setDatoNombre(nombre)
            setDatoApellido(apellido)
            setDatoCorreo(correo)
        }
    }

    return(
        <div>
            <h2>Formulario de Solicitudes (POST)</h2>
            <input type="text" placeholder="Ingrese el nombre" value={nombre} onChange={(e) => setNombre(e.target.value)} />
            <input type="text" placeholder="Ingrese el apellido" value={apellido} onChange={(e) => setApellido(e.target.value)} />
            <input type="email" placeholder="Correo" value={correo} onChange={(e) => setCorreo(e.target.value)} />
            <button onClick={Api}>Enviar</button>

            <div>
                <p>Nombre recibido: {datoNombre}</p>
                <p>Apellido recibido: {datoApellido}</p>
                <p>Correo recibido: {datoCorreo}</p>
                <p>Mensaje: {mensaje}</p>
            </div>
        </div>
    )
}
```

Backend


Explorer
Backend
node_modules
app.js
package-lock.json
package.json
app.js
app.js > ...
1 const express = require("express")
2 const cors = require("cors")
3 const app = express()
4 const PORT = 5000
5
6 app.use(express.json())
7 app.use(cors({
8 origin: "http://localhost:5173"
9 }))
10
11 app.post("/solicitudes", (req, res) => {
12 const { nombre, apellido, correo } = req.body
13
14 if (!nombre || !apellido || !correo) {
15 return res.status(400).json({ error: "Todos los campos son obligatorios" })
16 }
17
18 if (!correo.includes("@")) {
19 return res.status(400).json({ error: "Escribe un correo valido" })
20 }
21
22 res.json({
23 mensaje: "¡Datos recibidos correctamente!",
24 datosRecibidos: {
25 nombre,
26 apellido,
27 correo
28 }
29 })
30 })
31
32 app.listen(PORT, () => {
33 console.log("Servidor en puerto " + PORT)
34 })

7. Dificultades y Soluciones

6. Bloqueo por Políticas de CORS:

- **Problema:** El navegador rechazaba las peticiones de React hacia el puerto 5000 lanzando un error de CORS en la consola.

- **Solución:** Se instaló el paquete cors en Express (npm i cors) y se agregó app.use(cors()) en el archivo app.js permitiendo el acceso desde http://localhost:5173

Conclusión

Esta actividad me permitió afianzar el flujo completo de comunicación entre un cliente y un servidor. Aprendí la importancia de manejar estados reactivos en React, validar adecuadamente el estado de las peticiones HTTP (response.ok), estructurar formularios interactivos y configurar middleware de seguridad como CORS en Express.